

Java Backend Architecture

Interview Series

Chapter 14.10

System Design

Walkthroughs Part 2

50+ Interview Q&A

Code Examples

Architecture Diagrams

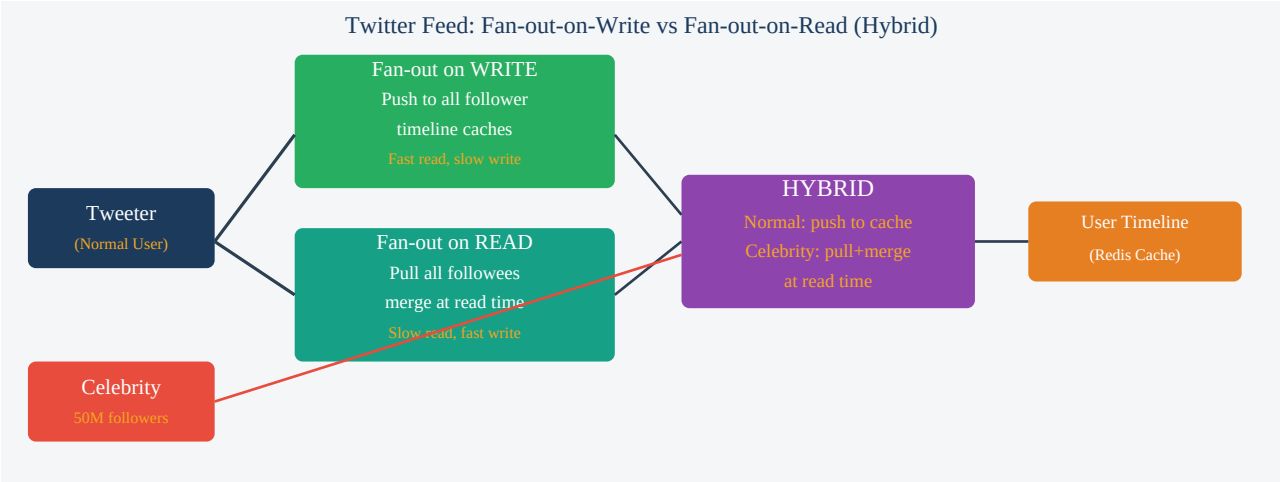
Advanced Topics

Chapter 14.10 - System Design Walkthroughs Part 2

Overview

This chapter covers four more system design walkthroughs: Twitter Feed (fan-out patterns), Hotel/Ticket Booking (seat locking and double-booking prevention), Uber/Ride-sharing (geospatial indexing), and Chat System (WebSocket, message ordering, read receipts).

1. Twitter Feed / Social Media Timeline



Fan-out Strategies

Strategy	Write Path	Read Path	Trade-off
Fan-out on Write (Push)	Push tweet to all followers' timeline caches on write	Read pre-built timeline from Redis (fast)	Fast reads, expensive writes for popular users
Fan-out on Read (Pull)	Write tweet to author's feed only	Pull from all followees, merge by time (slow)	Fast writes, slow reads, harder to cache
Hybrid	Push for normal users; pull for celebrities	Pre-built + celebrity tweets	Best balance, celebrity threshold (e.g., 1M followers)

Twitter Feed Implementation

```
// Post tweet (fan-out-on-write for normal users) @Service public class TweetService { private static final int CELEBRITY_THRESHOLD = 1_000_000; public void postTweet(UUID userId, String content) { Tweet tweet = tweetRepository.save(new Tweet(userId, content)); long followerCount = followService.getFollowerCount(userId); if (followerCount < CELEBRITY_THRESHOLD) { // Fan-out-on-write: push to all followers' timelines fanoutExecutor.submit() -> { List<UUID> followers = followService.getFollowers(userId); for (UUID followerId : followers) { String key = "timeline:" + followerId; redisTemplate.opsForZSet().add(key, tweet.getId().toString(), tweet.getCreatedAt().toEpochMilli()); redisTemplate.opsForZSet().removeRange(key, 0, -801); // keep 800 } }}; } // Celebrity tweets are pulled at read time } // Read timeline (hybrid) public List<Tweet> getTimeline(UUID userId, int page) { // Pre-built timeline from Redis (non-celebrity followees) Set<String> tweetIds = redisTemplate.opsForZSet().reverseRange("timeline:" + userId, page * 20L, (page + 1) * 20L - 1); // Pull celebrity tweets and merge List<UUID> celebrities = followService.getCelebrityFollowees(userId); List<Tweet> celebTweets = tweetRepository.findByCelebsAfter(celebrities, lastSeenTimestamp); return merge(fetchTweets(tweetIds), celebTweets); }
```

Feed Storage Design

Timeline cache	Redis sorted set per user: key="timeline:{userId}", member=tweetId, score=timestamp. Trim to last 800 tweets. ZRevRange for pagination.
Tweet store	Primary DB table: tweets(id, user_id, content, created_at). Read by ID for rendering. MySQL/Cassandra depending on write volume.
Social graph	followers(follower_id, followee_id, created_at). Index on both columns. Cache celebrity follower lists in Redis (large lists).
Celebrity list	Precomputed list of celebrity user IDs (> threshold). Updated on follower count change. Checked at read time for hybrid approach.

2. Hotel / Ticket Booking System

Hotel Booking: Seat Locking Sequence



Double-Booking Prevention Techniques

1. SELECT FOR UPDATE (row lock)
2. UNIQUE constraint on (room_id, date) for BOOKED status
3. Optimistic lock (version column CAS)
4. Cron job to expire HELD rooms after timeout

Double-Booking Prevention Strategies

Status + CAS	UPDATE rooms SET status=HELD, held_by=user, held_until=now+15m WHERE id=? AND status=AVAILABLE. Only one transaction wins (atomic). Returns 0 rows affected = already taken.
SELECT FOR UPDATE	SELECT * FROM rooms WHERE id=? FOR UPDATE acquires row lock. Only one transaction can hold the lock. Others wait. Prevents concurrent booking of same row.
UNIQUE constraint	bookings(room_id, date) UNIQUE ensures DB rejects duplicate booking at DB level even if app logic fails.
Optimistic lock	rooms has version column. UPDATE SET status=HELD, version=version+1 WHERE id=? AND version=? AND status=AVAILABLE. Returns 0 rows -> retry.
Cron cleanup	@Scheduled job every 5 minutes: UPDATE rooms SET status=AVAILABLE WHERE status=HELD AND held_until < NOW(). Releases expired holds.
Idempotency key	Payment endpoint accepts idempotency key (UUID from client). Store in payments(idempotency_key UNIQUE). Same key = return existing result, not re-charge.

Booking Service Code

```

@Service @Transactional public class BookingService {
    public BookingResult holdRoom(Long roomId, UUID userId, LocalDate date) {
        // Atomic CAS update - only one winner
        int updated = roomRepository.holdRoom(roomId, userId, LocalDateTime.now().plusMinutes(15));
        if (updated == 0) {
            throw new RoomNotAvailableException("Room " + roomId + " already held");
        }
        Booking booking = bookingRepository.save(new Booking(roomId, userId, date, BookingStatus.HELD));
        return new BookingResult(booking.getId(), BookingStatus.HELD);
    }

    public void confirmBooking(UUID bookingId, String idempotencyKey) {
        // Idempotency check
        if (paymentRepository.existsByIdempotencyKey(idempotencyKey)) {
            return; // already processed
        }
        Booking booking = bookingRepository.findById(bookingId).orElseThrow(() -> new
            NotFoundException("Booking not found"));
        if (booking.getStatus() != BookingStatus.HELD) {
            throw new IllegalStateException("Booking not in HELD state");
        }
        paymentService.charge(booking, idempotencyKey);
        booking.setStatus(BookingStatus.CONFIRMED);
        roomRepository.setBooked(booking.getRoomId());
    }
}

// Repository - atomic CAS update
@Query("UPDATE Room r SET r.status = HELD, r.heldBy = :userId, r.heldUntil = :until WHERE r.id
  
```

```
= :id AND r.status = AVAILABLE") @Modifying int holdRoom(@Param("id") Long id, @Param("userId")  
UUID userId, @Param("until") LocalDateTime until);
```

3. Uber / Ride-sharing System Design



Geospatial Indexing

S2 Cells	Google S2 library divides the globe into a hierarchy of cells at multiple levels (level 12 ~1km ² , level 14 ~0.15km ²). Each cell has a 64-bit ID. Find nearby drivers: compute S2 cells covering a radius, query drivers in those cells.
H3 Hexagons	Uber H3: hexagonal hierarchical spatial indexing. Hexagons have equal area, equal neighbor distance (better than rectangular grids). 15 resolution levels. h3.geoToH3(lat, lng, resolution) returns cell ID.
Redis GEO	Redis GEOADD key lon lat member. GEORADIUS/GEORADIUSBYMEMBER returns members within radius. Internally uses geohash encoded as sorted set score. Simpler than S2/H3 but less flexible for hierarchical queries.
Driver location	Driver app sends location update (WebSocket or HTTP) every 4-5 seconds. Location service updates Redis GEO set and the driver's cell index.
Trip matching	Rider requests trip: find all available drivers in rider's cell + adjacent cells. Filter by: availability, direction, ETA, rating. Assign closest available driver.

Geospatial Location Service

```
// Driver location update @Service public class LocationService { public void
updateDriverLocation(String driverId, double lat, double lon) { // Redis GEO index for
proximity queries redisTemplate.opsForGeo().add("drivers:available", new Point(lon, lat),
driverId); // H3 cell for cell-based lookup long cellId = H3Core.getInstance().geoToH3(lat,
lon, 9); // ~500m resolution redisTemplate.opsForSet().add("cell:" + cellId, driverId); //
Expiry: driver considered offline if no update in 30s redisTemplate.expire("driver:loc:" +
driverId, 30, TimeUnit.SECONDS); } public List<DriverDto> findNearbyDrivers(double lat, double
lon, double radiusKm) { // Redis GEORADIUS GeoResults<GeoLocation<String>> results =
redisTemplate.opsForGeo().radius("drivers:available", new Circle(new Point(lon, lat), new
Distance(radiusKm, Metrics.KILOMETERS)),
GeoRadiusCommandArgs.newGeoRadiusArgs().includeDistance().limit(10)); return
results.getContent().stream().map(r -> new DriverDto(r.getContent().getName(),
r.getDistance().getValue())) .collect(toList()); } }
```

WebSocket for Real-time Driver Tracking

```
// WebSocket handler @Component public class DriverLocationHandler extends
TextWebSocketHandler { private final Map<String, WebSocketSession> driverSessions = new
ConcurrentHashMap<>(); @Override public void afterConnectionEstablished(WebSocketSession
session) { String driverId = extractDriverId(session); driverSessions.put(driverId, session);
} @Override protected void handleTextMessage(WebSocketSession session, TextMessage msg) {
LocationUpdate update = objectMapper.readValue(msg.getPayload(), LocationUpdate.class);
locationService.updateDriverLocation(update.getDriverId(), update.getLat(), update.getLon());
} // Push trip assignment to driver public void sendTripAssignment(String driverId, TripRequest
trip) throws Exception { WebSocketSession session = driverSessions.get(driverId); if (session
!= null && session.isOpen()) { session.sendMessage(new TextMessage(
objectMapper.writeValueAsString(trip))); } } }
```

4. Chat System Design (WhatsApp-style)

Chat System Architecture

WebSocket	Persistent bidirectional connection between client and chat server. Server can push messages without polling. Connection affinity: user always connects to the same chat server (sticky sessions or Redis Pub/Sub for cross-server delivery).
Message storage	messages(id BIGINT, conversation_id, sender_id, content, created_at, type). Cassandra: partition by conversation_id, cluster by message_id for efficient range reads. Or HBase for massive scale.
Message ordering	Use a monotonically increasing sequence per conversation (Snowflake ID or DB sequence). Client displays by sequence number, not client timestamp (clocks are not synchronized).
Read receipts	message_reads(message_id, user_id, read_at). Sender subscribes to read receipt events. Update status: SENT -> DELIVERED (server received) -> READ (recipient opened).
Group chat fan-out	On message sent to group: fan out to all member conversations (similar to Twitter feed). For large groups (1000+ members): use a fan-out service with Kafka. Store group membership in a separate table.
Offline delivery	If recipient is offline, store message with PENDING status. On reconnect, server sends all pending messages. Push notification (FCM) to wake the app for foreground delivery.

Chat Message Service

```
// Message sending @Service public class ChatService { public Message sendMessage(UUID
senderId, UUID conversationId, String content) { // Assign sequence number (per conversation)
long seqNum = redisTemplate.opsForValue().increment("seq:" + conversationId); Message msg =
messageRepository.save( new Message(conversationId, senderId, content, seqNum, Instant.now())
); // Fan-out to conversation members List<UUID> members =
conversationRepo.getMembers(conversationId); for (UUID memberId : members) { if
(!memberId.equals(senderId)) { deliverMessage(memberId, msg); } } return msg; } private void
deliverMessage(UUID userId, Message msg) { // Try WebSocket delivery (if user is online)
WebSocketSession session = sessionRegistry.get(userId); if (session != null &&
session.isOpen()) { websocketService.send(session, msg); updateDeliveryStatus(msg.getId(),
userId, DeliveryStatus.DELIVERED); } else { // User offline: queue for later + push
notification pendingMessageRepo.save(new PendingMessage(userId, msg.getId()));
pushService.sendSilentPush(userId, "New message from " + msg.getSenderName()); } } // Mark as
read public void markRead(UUID userId, UUID conversationId, long upToSeqNum) {
messageReadRepo.upsert(userId, conversationId, upToSeqNum, Instant.now()); // Notify sender of
read receipt eventPublisher.publish(new ReadReceiptEvent(userId, conversationId, upToSeqNum));
} }
```

Multi-Server Chat Delivery with Redis Pub/Sub

```
// Cross-server message delivery // Problem: User A connected to Server1, User B connected to
Server2 // Solution: Redis Pub/Sub channel per user @Component public class ChatMessageBroker {
public void deliverToUser(UUID userId, Message msg) { String channel = "user:msg:" + userId; //
Publish to Redis channel - all chat servers listen redisTemplate.convertAndSend(channel,
objectMapper.writeValueAsString(msg)); } @PostConstruct public void subscribe() { // Each
server subscribes to messages for its connected users
```

```
redisMessageListenerContainer.addMessageListener( (message, pattern) -> { String userId =  
extractUserId(message.getChannel()); WebSocketSession session = sessionRegistry.get(userId);  
if (session != null) { session.sendMessage(new TextMessage( new String(message.getBody()))); }  
}, new PatternTopic("user:msg:*") ); } }
```

Interview Q&A - System Design Walkthroughs Part 2

Q1. Explain fan-out-on-write vs fan-out-on-read for a social media feed.

Fan-out-on-write (push): when a user posts, immediately push the post to all followers' precomputed timeline caches. Read is fast (just read cache). Write is expensive (N followers = N cache updates). Fan-out-on-read (pull): post is stored once; at read time, pull from all followees and merge. Write is fast; read is slow and hard to cache. Hybrid (Twitter): push for normal users, pull-at-read for celebrities (millions of followers would make push prohibitively expensive).

Q2. What is the celebrity problem in social media feed design?

A celebrity with 50 million followers triggering fan-out-on-write would create 50 million Redis writes and Kafka messages for each tweet. This is a "hotspot" write amplification. Solutions: (1) Hybrid approach - only fan-out-on-write for users below a follower threshold (e.g., 1 million); (2) For celebrities, pull their latest N tweets at read time and merge with the pre-built timeline; (3) Batch fan-out - process celebrity fan-outs asynchronously over minutes/hours.

Q3. How do you store and page through a Twitter-style timeline?

Redis Sorted Set: key="timeline:{userId}", member=tweetId, score=tweetTimestamp (Unix ms). ZADD to add tweets (fan-out), ZREVRANGE timeline:userId 0 19 for first 20 tweets (cursor-based), ZREVRANGEBYSCORE for cursor pagination. Trim to last 800 entries (ZREMRANGEBYRANK 0 -801) to bound memory. For tweets older than the cache window, fall back to DB query. TTL the cache key to evict inactive users.

Q4. How do you prevent double-booking in a hotel reservation system?

Atomic status CAS: UPDATE rooms SET status=HELD, held_by=userId, held_until=now+15min WHERE id=? AND status=AVAILABLE. Only one concurrent transaction wins; others get 0 rows affected and receive "unavailable". Additionally: UNIQUE constraint on bookings(room_id, checkin_date) for status=BOOKED as DB-level safety net. SELECT FOR UPDATE for read-modify-write flows. Idempotency key on payment to prevent duplicate charges on retry.

Q5. What is seat locking and how long should the lock last?

Seat locking temporarily marks a seat as HELD to give the user time to complete payment. The lock duration must balance: long enough for payment (2-3 min for card entry) vs short enough to release quickly if abandoned. Typically 10-15 minutes. Implementation: update seat status to HELD with held_until timestamp. A scheduled job (every 1-5 minutes) scans for expired holds and resets to AVAILABLE. Redis can also store the hold with TTL for fast cleanup.

Q6. How do you handle payment idempotency in a booking system?

The client generates a unique idempotency key (UUID) before the payment request. The payment endpoint stores the key atomically with the payment result (UNIQUE constraint on idempotency_key). If the same key is received again (retry after timeout/network failure), return the previously stored result without re-charging. This ensures that network retries never result in double charges. AWS, Stripe, and Braintree all support this natively.

Q7. How do geospatial indices like S2 or H3 work for ride-sharing?

The globe is divided into a hierarchical grid of cells. S2 cells are square-ish (cube face projections); H3 cells are hexagonal (better for proximity - all neighbors at equal distance). Each cell has a unique integer ID.

Driver's lat/lon maps to a cell ID (h3.geoToH3(lat, lon, resolution)). To find nearby drivers: compute the cell ID and its neighbors (h3.kRing(cellId, 1) = 7 cells including center). Query drivers assigned to those cells.

Q8. How does Redis GEO work for location-based queries?

Redis GEO commands use a geohash encoding (52-bit integer) stored in a sorted set (score = geohash). GEOADD key lon lat member adds a location. GEORADIUS/GEORADIUSBYMEMBER finds all members within a radius. GEOPOS returns coordinates. GEODIST calculates distance between two members. Internally it uses a Z-order curve (geohash) which groups nearby coordinates into similar integers, enabling efficient range scans on the sorted set for proximity queries.

Q9. How frequently should drivers update their location and why?

Every 4-5 seconds is the industry standard (Uber uses ~5s). Too frequent: battery drain on driver's phone, excessive Redis writes (at 1M drivers, 5s interval = 200,000 writes/sec), network overhead. Too infrequent: rider sees jerky, inaccurate driver position on map, matching uses stale location. During trip: update more frequently (1-2s) for smooth map animation. When idle/between trips: less frequently (10-30s) to save battery.

Q10. What is WebSocket and when should you use it vs HTTP polling?

WebSocket is a full-duplex, persistent TCP connection upgraded from HTTP. After the HTTP upgrade handshake, both client and server can send frames at any time without new connections. Use WebSocket for: real-time bidirectional communication (chat, live location, collaborative editing, trading prices), where push semantics are needed. Use HTTP polling (or SSE for server-push only) for: infrequent updates, firewall-restricted environments, or simple read-heavy applications where polling interval matches update frequency.

Q11. How do you handle WebSocket connections at scale?

WebSocket connections are persistent and stateful - a user must stay connected to the same server instance. Horizontal scaling strategy: sticky sessions (load balancer routes user to same server by session cookie or IP hash). For message delivery across servers: Redis Pub/Sub - each server subscribes to channels for its connected users; message is published to the channel and delivered by whichever server holds the connection. Server-sent Events (SSE) is simpler for server-to-client push (half-duplex).

Q12. How do you ensure message ordering in a chat system?

Use a monotonically increasing sequence number per conversation. Options: (1) Redis INCR on "seq:{conversationId}" - atomic, fast, but Redis dependency. (2) Database sequence or auto-increment per conversation partition. (3) Snowflake IDs (timestamp-based, lexicographically sortable). Display messages sorted by sequence number (server-assigned), not client timestamp (unreliable due to clock skew). On client: buffer incoming messages and display in sequence order.

Q13. How do read receipts work in a chat system?

Read receipts track who has seen each message. Schema: message_reads(message_id, user_id, read_at). When user opens a conversation: mark all unread messages as READ (upsert in batch for efficiency). Server sends READ receipt event back to the sender via WebSocket. Sender's client updates message status icon (single tick = sent, double tick = delivered, blue double tick = read). Group chat: track read status per member. Only show read receipt if sender has read receipts enabled (privacy).

Q14. How does fan-out work for a group chat with 500 members?

When a message is sent to a group: (1) Save message to DB. (2) Fan-out: publish MessageEvent to Kafka. (3) Fan-out service consumes and iterates over 500 members: for each online member, deliver via

WebSocket (via Redis Pub/Sub if on a different server); for offline members, store pending message and send push notification. Rate-limit push notifications for active groups. For very large groups (5,000+), consider inbox fan-out: on read, query the last N group messages.

Q15. How do you handle offline message delivery in a chat system?

When a message is sent and the recipient is offline: store in pending_messages(recipient_id, message_id, created_at). Send a silent push notification (FCM/APNs data-only message) to wake the app. On app launch/reconnect, the client sends its last sequence number; server queries all messages with seq > last_seen and delivers in order. Clear pending_messages after delivery. Limit pending messages per user (e.g., 1,000) to avoid unbounded storage.

Q16. What is the difference between ephemeral (Snapchat) and persistent (WhatsApp) messages?

Ephemeral: messages are deleted after viewing or after a TTL. Store in Redis with TTL or in DB with delete-after-read logic. Screenshot detection (notify sender). No message history after expiry. Persistent: messages stored in DB indefinitely (or until user deletes). Full conversation history. Multi-device sync: new device can fetch full history. End-to-end encryption (WhatsApp): server never stores decrypted content; encrypted ciphertext is stored; only endpoints have keys.

Q17. How do you design the database for a chat system?

Cassandra is well-suited: partition by conversation_id, cluster by message_id (Snowflake, time-sorted). Query pattern: get last N messages = single partition scan (efficient). Write pattern: append-only (Cassandra optimized for writes). Schema: messages(conversation_id UUID, message_id TIMEUUID, sender_id UUID, content TEXT, type TEXT, PRIMARY KEY (conversation_id, message_id)) WITH CLUSTERING ORDER BY (message_id DESC). Index on sender_id for "find all messages by user" queries.

Q18. How does trip matching work in a ride-sharing system?

When rider requests: (1) Find all available drivers within radius (Redis GEORADIUS or H3 cell query). (2) Filter by: status=AVAILABLE, rating > threshold, car type matches request, heading in compatible direction. (3) Rank by: ETA (estimated time of arrival = route calculation), driver rating. (4) Send trip request to top N drivers simultaneously (first to accept wins). (5) On acceptance: update driver status=ON_TRIP, notify rider with ETA. Timeout: if no driver accepts in 30s, expand search radius.

Q19. What is surge pricing and how is it calculated?

Surge pricing increases fares when demand exceeds supply in a geographic area. Demand = ride requests per H3 cell in last 5 minutes. Supply = available drivers per cell. Surge multiplier = f(demand/supply ratio). Buckets: 1.0x (normal), 1.2x-1.5x (light surge), 2.0x-3.0x (heavy surge). Data pipeline: aggregate real-time demand/supply metrics per H3 cell using Kafka Streams or Flink. Notify riders of surge before booking. Cap multiplier (e.g., 5x) for PR/regulatory compliance.

Q20. How do you handle the "thundering herd" problem when a popular tweet is posted?

When a celebrity posts (millions of followers), many read requests hit the timeline simultaneously. Mitigation: (1) Lazy fan-out: do not write to all follower caches immediately; write celebrity tweet to a separate celebrity feed store; at read time, merge. (2) Cache warming: gradually warm follower caches before the tweet is "announced". (3) Rate-limit cache misses: only one request triggers DB fetch; others wait (Singleflight pattern in Go, or Redis distributed lock). (4) Pre-compute celebrity feeds on a schedule.

Q21. How do you handle concurrent seat booking for a concert ticketing system?

Two-phase lock approach: (1) Show available seats (SELECT WHERE status=AVAILABLE). (2) User selects seats; initiate hold: UPDATE tickets SET status=HELD, user_id=? WHERE id IN (?) AND status=AVAILABLE. Count rows updated - if < requested count, some seats were taken; rollback. (3) User completes payment within hold window (10-15 min). (4) Confirm: UPDATE SET status=SOLD. Distributed lock (Redis SETNX) can additionally prevent concurrent selection of the same seat IDs during the UI selection phase.

Q22. What is geohash and how does it relate to S2/H3?

Geohash encodes lat/lon as a string of characters (e.g., "u4pruydqqvj") by interleaving bits of latitude and longitude in a Z-order curve. Nearby locations share long common prefixes, enabling prefix-search for proximity queries. Geohash is simpler but has edge distortion (cells near poles are smaller). S2 uses a sphere-to-cube projection for more uniform cell sizes. H3 uses hexagonal grid for equal neighbor distances. Redis GEO uses 52-bit geohash internally.

Q23. How do you scale the location update ingestion service at Uber scale?

At 5M concurrent drivers updating every 5s = 1M location updates/sec. Architecture: drivers connect via WebSocket to location ingestion service (stateless, horizontally scaled). Updates written to Kafka (high-throughput, durable). Location processor consumes from Kafka: updates Redis GEO (current position), H3 cell index, driver status table. Kafka handles burst (buffering), Redis handles low-latency queries. Redis cluster with sharding by driverId for writes; GEORADIUS for reads.

Q24. How do you implement typing indicators in a chat system?

When user starts typing: send a "typing" event to the server (over existing WebSocket). Server broadcasts to other conversation participants via Redis Pub/Sub or direct WebSocket delivery. Client displays "User X is typing..." indicator. Events expire after 3-5 seconds (no explicit "stopped typing" needed - the indicator auto-hides). Throttle typing events: send only when state changes or every 3s (not on every keystroke). Do not persist typing events to DB.

Q25. What is the message delivery guarantee in a chat system?

At-least-once delivery: messages may be delivered more than once; client deduplicates by message_id. Exactly-once: much harder - requires idempotent delivery and client-side dedup. In practice: use at-least-once semantics with client-side deduplication (track seen message IDs in a client-side set or by sequence number). WebSocket frame loss: implement ACK protocol - client ACKs each message; server retransmits unACKed messages after timeout.

Q26. How do you design a booking system for multiple booking types (hotel, flight, concert)?

Generic reservation pattern: items(id, type ENUM(HOTEL_ROOM, FLIGHT_SEAT, CONCERT_TICKET), status ENUM(AVAILABLE, HELD, BOOKED), held_by, held_until, version). Core booking logic is the same (CAS status update). Item-specific details in separate tables joined by item_id. Pricing engine, availability calendar, and cancellation policies are item-type-specific plugins/strategies. The Booking Service handles the common flow; delegates to item-specific services for availability check and confirmation.

Q27. How do you ensure WebSocket message ordering with multiple server instances?

Messages sent via Redis Pub/Sub may arrive out of order if published rapidly. Solutions: (1) Sequence numbers - client buffers and reorders by seq; request missing sequences from server. (2) Single server per conversation - consistent hashing routes all conversation messages to the same server instance; no cross-server ordering issues. (3) Kafka partitioned by conversation_id - ordered delivery within partition; fan-out service delivers in order.

Q28. What is connection draining for WebSocket servers?

When deploying a new version of the WebSocket server, simply killing the process disconnects all users. Connection draining: (1) Mark the server as draining (stop accepting new connections). (2) Send a "please reconnect" message to all connected clients. (3) Clients reconnect to the load balancer, which routes to healthy instances. (4) Wait for all existing connections to close (or force close after timeout). Spring Boot supports graceful shutdown for WebSocket via `server.shutdown=graceful`.

Q29. How does the hotel booking system handle overbooking?

Overbooking (selling more rooms than available) can be intentional (airlines do this - expected no-shows) or accidental. To prevent accidental overbooking: UNIQUE constraint + CAS status update (as described). For intentional overbooking: track an `overbooking_limit` per `room_type`. Allow bookings up to `available + overbook_limit`. Run compensation flow: if all rooms are filled and overbooking limit reached, offer upgrade or refund with compensation to last-booked customers.

Q30. How do you design the data model for Uber trip history?

trips(id UUID PK, rider_id, driver_id, pickup_lat, pickup_lon, dropoff_lat, dropoff_lon, start_time, end_time, distance_km, fare, status, route_polyline). Indexes: rider_id + start_time (user trip history), driver_id + start_time (driver earnings). Partition trips by month for large tables (PostgreSQL range partitioning). Archive trips > 1 year to cold storage (S3 with Athena for analytics). Real-time trip tracking: Kafka for location events during trip; GPS polyline stored on trip completion.

Q31. How do you handle timezone issues in a booking/reservation system?

Store all times in UTC in the database. Convert to/from local timezone in the application layer. For hotel check-in: store date only (LOCAL_DATE), not datetime, since "check-in on Jan 5" means Jan 5 in the hotel's timezone regardless of the booker's timezone. For flight bookings: store departure/arrival as UTC datetime + airport timezone. Always display to users in their local timezone. Java: use `ZonedDateTime` for exact moments, `LocalDate` for date-only concepts like check-in.

Q32. How do you implement end-to-end encryption (E2EE) in a chat system?

Signal Protocol (used by WhatsApp, Signal): each user has a public/private key pair stored on their device (private key never leaves device). Key exchange (X3DH): establishes a shared secret between two users. Messages are encrypted with the shared secret before sending. Server stores only ciphertext - cannot decrypt. Multi-device: each device has its own key; sender encrypts message once per recipient device. Key verification: compare safety numbers/QR code to detect man-in-the-middle.

Q33. What is the difference between long polling, SSE, and WebSocket?

Long polling: client sends HTTP request; server holds it open until data is available, then responds. Client immediately sends a new request. Simple, works everywhere, but high overhead (new HTTP connection per update). SSE (Server-Sent Events): one-way (server to client) persistent HTTP stream. Client reconnects automatically. Good for: notifications, live feeds. WebSocket: full-duplex, low overhead (frames, not full HTTP requests). Best for: chat, real-time collaboration, gaming, location tracking.

Q34. How do you handle the case where a driver cancels during a trip?

Trip cancellation flow: driver sends cancel event -> driver location service marks driver AVAILABLE again -> trip status = CANCELLED -> rider is notified via WebSocket/push. Re-match: automatically search for new drivers near rider's current location. Compensation: if cancelled after pickup, apply cancellation fee to driver. Rate limit: too many cancellations deactivates driver. Rider-facing: show re-matching UI with estimated wait time. Log cancellation reason for driver quality tracking.

Q35. How does message search work in a chat application?

Elasticsearch for full-text message search. Index message content at write time (Kafka consumer writes to ES). Search query: full-text search with conversation scope filter. Privacy: index only messages the requesting user has access to (filter by conversation membership). Retention: ES index may have shorter retention than primary DB (search within last 30 days vs full history in Cassandra). Highlight matching terms in search results. Use ES's more-like-this query for "find similar messages".

Q36. What is path finding and ETA calculation in a ride-sharing app?

ETA = distance / average_speed adjusted for real-time traffic. Approaches: (1) Google Maps Distance Matrix API / Mapbox Matrix API for accurate routing. (2) Internal routing engine (OSRM, Valhalla) for cost control at scale. Input: origin, destination, time-of-day. Output: ETA in minutes, route polyline. Cache frequently computed routes (origin-destination pairs near popular pickup zones). Update ETA in real-time as driver deviates from route (compare GPS track to planned route every 10s).

Q37. How do you implement presence (online/offline status) in a chat system?

When user connects via WebSocket: set presence key in Redis (SETEX "presence:{userId}" 30 "online"). Heartbeat: client sends ping every 15s; server updates TTL (EXPIRE "presence:{userId}" 30). On disconnect: delete key immediately. To check if a user is online: EXISTS "presence:{userId}". Broadcast status changes: when a user goes online/offline, publish to "presence:{userId}" Pub/Sub channel; friends' clients subscribed via WebSocket receive the status change.

Q38. How does hotel search work for a given date range?

Room availability for a date range: a room is available if no booking overlaps the requested dates. SQL: SELECT r.* FROM rooms r WHERE r.hotel_id=? AND r.type=? AND NOT EXISTS (SELECT 1 FROM bookings b WHERE b.room_id=r.id AND b.status IN (HELD, CONFIRMED) AND b.checkin_date < :checkout AND b.checkout_date > :checkin). Index on (room_id, checkin_date, checkout_date). For high-traffic hotel search: pre-compute availability calendar in Redis bitmap (1 bit per day per room).

Q39. What is the CQRS pattern applied to the Twitter feed?

Command side: POST /tweets (write model) - saves tweet to DB, publishes TweetCreated event. Query side: GET /timeline/{userId} (read model) - reads from pre-built Redis timeline cache (denormalized, sorted list of tweetIds). The fan-out service acts as the projection: consumes TweetCreated events and updates the read model (Redis sorted sets) for all followers. Classic CQRS: command path is optimized for writes; query path is optimized for reads with a denormalized, pre-computed timeline.

Q40. How do you handle message encryption keys for a group chat in E2EE?

For 1-1 chats: Signal X3DH key exchange between two parties. For group chats: sender generates a symmetric group key, encrypts it separately for each member's public key, and sends the encrypted keys alongside the message (sender key distribution). New member joins: existing members must re-encrypt the current key for the new member. Member leaves: key rotation required (generate new key, re-encrypt for remaining members) to ensure forward secrecy.

Q41. What is the role of Kafka in a chat system?

Kafka provides: (1) Message durability: if notification service is down, messages are not lost. (2) Fan-out: a single message event consumed by multiple services (notification, analytics, search indexer). (3) Ordering: messages per conversation partitioned by conversation_id - ordered delivery. (4) Replay: reindex lost data by replaying messages. (5) Decoupling: chat service produces events; delivery, push, analytics services

consume independently. Topic: chat-messages partitioned by conversation_id.

Q42. How do you design a booking system with multiple concurrent users selecting seats?

The key challenge: seats are selected in UI (optimistic), then locked on submit. To avoid poor UX (selected seat appears available but fails on submit): Step 1: Soft lock at UI level (show seat as "being held" after 30s of selection - purely informational). Step 2: Hard lock on "Proceed to Payment" button: acquire DB hold immediately with short TTL (5 min). Step 3: Payment must complete within hold window. This reduces lock duration to only the payment window vs the entire selection time.

Q43. How do you handle geofencing in a ride-sharing system?

Geofencing defines a virtual boundary (airport, downtown zone, restricted area). Use cases: surge pricing zones, driver restricted zones, airport pickup rules. Implementation: store geofence as polygon (list of lat/lon points) or circle (center + radius). To check if a point is inside: point-in-polygon algorithm (ray casting). For real-time: on each driver location update, check if driver entered/left any geofence (using H3 cell membership in geofence cell set). Redis GEO + polygon membership check.

Q44. What is conversation/message pagination and how is it cursor-based?

Cursor-based pagination avoids the "page N" instability problem (new messages shift page boundaries). Use message sequence number as cursor. First load: GET /conversations/{id}/messages?limit=20 returns messages ordered by seq DESC, last message seq as "nextCursor". Load more (older): GET /messages?cursor=1234&limit=20 returns messages where seq < cursor. This is stable - inserting new messages does not affect historical pages. Encode cursor as opaque string (base64) to hide internals.

Q45. How do you implement "last seen" timestamp in a chat system?

Track per-user last activity timestamp: UPDATE users SET last_seen=NOW() on WebSocket message receive or heartbeat. Display format: "Last seen today at 2:30 PM", "Last seen 2 days ago". Privacy settings: hide last_seen (return null). Cache last_seen in Redis (SETEX) for fast access - update DB asynchronously in batch to reduce write load (batch all last_seen updates every 30s via @Scheduled). Index users.last_seen for "find inactive users" admin queries.

Q46. How do you implement message reactions in a chat system?

Schema: message_reactions(message_id, user_id, emoji, created_at, PRIMARY KEY(message_id, user_id)). One reaction per user per message (upsert to change reaction). Aggregate reactions: SELECT emoji, COUNT(*) FROM message_reactions WHERE message_id=? GROUP BY emoji. Cache reaction counts in Redis (HINCRBY "reactions:{messageId}" emoji 1) for fast reads. WebSocket broadcast when reaction added/removed. For high-volume messages, batch reaction count updates to DB.

Q47. What is consistent geohashing and how does Uber use it for city-level architecture?

For global scale, Uber uses a city-based sharding strategy: each city (or geographic region) is a unit of sharding. All trip data for New York routes to the New York shard. Drivers in New York register only with the New York location service. Cross-city trips (airport on city boundary) are handled by a routing layer. This reduces cross-shard coordination for the 99% case (trips within one city). Consistent hashing determines which service instance handles a given city based on hashed city ID.

Q48. How do you handle concurrent updates to a chat group's member list?

Race condition: two admins simultaneously add different members. Use optimistic locking: groups table has version column. UPDATE groups SET member_count=member_count+1, version=version+1 WHERE id=? AND version=?. If version mismatch (0 rows updated), retry. For member list itself: store as separate table

group_members(group_id, user_id, role, joined_at). Add member is an INSERT - naturally idempotent with UNIQUE (group_id, user_id). Remove member is DELETE - idempotent. No locking needed for member list mutations.

Q49. How do you design the payment retry logic for a booking system?

If payment times out or fails with a transient error: retry with exponential backoff (1s, 2s, 4s, max 3 retries). Use the same idempotency key on each retry. If payment fails permanently (insufficient funds, card declined): release the seat hold (UPDATE status=AVAILABLE), notify user. If payment service is unreachable: keep hold, send push notification to complete payment (with deeplink to payment page). Implement a payment timeout job: if hold is > 30 min with no payment, auto-cancel.

Q50. What is the difference between home timeline and user timeline in Twitter-style systems?

Home timeline: the aggregated feed of tweets from accounts the user follows - computed via fan-out. User timeline: all tweets posted by a specific user - stored directly in that user's own feed table; simple query. Home timeline is the expensive, fan-out-computed view; user timeline is just the user's own posts sorted by time. Both are cached in Redis. User timeline is used on profile pages; home timeline is the main feed.

Q51. How does the ETA update during an active Uber trip?

On trip start: calculate initial ETA using routing API (origin to destination). Every 10s: compare driver's current GPS position to planned route. If driver deviates (traffic reroute, missed turn): recalculate ETA using current position as new origin. Update rider's app via WebSocket push ("Updated ETA: 12 min"). Track progress: how far along the planned route is the driver (reduce ETA as driver progresses). ETA accuracy is a key UX metric - monitor and tune the routing model.

Q52. What are the database trade-offs for storing chat messages - Cassandra vs PostgreSQL vs MySQL?

Cassandra: excellent for chat - high write throughput, partition by conversation_id gives fast range reads, scales horizontally, no schema-level joins needed. Best for: millions of conversations, billions of messages. PostgreSQL: fine for smaller scale (<10M messages), supports complex queries (full-text search, aggregations), ACID transactions. MySQL: similar to PostgreSQL, used by Facebook Messenger historically with heavy sharding. Key Cassandra advantage: linear write scalability and time-series access pattern alignment.
